

Och

1 Introduction

Static analysis catches bugs before software is deployed, notifying developers of mistakes earlier in the development pipeline than testing or user reports can. Type systems are the most widely adopted form of static analysis, and the strength of guarantees they provide varies enormously: from freedom from runtime type errors (TypeScript) through ownership-based memory safety (Rust) to full functional correctness (Lean, Agda, Coq). At the strong end of this spectrum lies *formal verification*: a compiler-checked mathematical proof that a program satisfies its specification across all possible inputs.

Formal verification has been applied at scale: a verified C compiler (Leroy 2009), a verified operating system kernel (Klein et al. 2009), and a verified HTTPS stack (Bhargavan et al. 2017) all demonstrate that proving the correctness of real systems software is feasible. These projects used a variety of proof technologies, but a common thread among the most ergonomic approaches is *dependent types*: a type-theoretic mechanism by which types may depend on values, allowing the programmer to state properties such as “this function returns a list of the same length as its input” directly in a type signature, with the compiler verifying the property as a condition of successful compilation.

There is a large intersection between i) software which requires low-level control and performance, and ii) software where correctness is critical. Operating system kernels, cryptographic libraries, network protocol implementations, and compiler backends all occupy this intersection — they are *systems software*. Today, the languages which offer the necessary control (C, C++, Rust) lack the type-theoretic machinery for formal verification, while the languages which support dependent types (Lean, Agda, F*) lack the control over memory layout, allocation, and mutation that systems programming demands.

Ochre is a planned language which aims to inhabit this gap: a systems programming language with native support for dependent types. One might summarise the goal as combining Rust’s ownership model and low-level control with Lean’s proof capabilities in a single language and type system. The key mechanism that enables Ochre is the isomorphism between (safe) Rust and Lean (Ho and Protzenko 2022) — if we can translate freely between Rust and Lean, and we know how to dependently type check Lean, then we must be able to dependently type check Rust.

Two recent trends make this research direction timely. First, the cost of unverified software is rising: AI-assisted vulnerability discovery has dramatically increased the rate at which security bugs are found in critical infrastructure — Mozilla closed as many Firefox vulnerabilities in April 2026 as in the previous two years combined (Knop 2026), and a nine-year dormant Linux kernel privilege escalation was surfaced by automated analysis (Microsoft Security Response Center 2026). Second, the cost of writing software is falling: AI coding agents have reduced the labour involved in software engineering to the point where ambitious rewrites that were previously infeasible are now routine (Sumner 2026). The bottleneck for software engineering is shifting from writing code to specifying what correct code looks like; the value proposition of a language whose type system *is* a specification language is growing stronger.

During a previous attempt to develop Ochre (Lidbury 2024), the project was held back by soundness bugs in the type system and a general lack of clarity on the semantics of the objects in the language. A counterexample was found in which narrowing a variable’s type — the operation which underlies strong mutation — invalidated a previously valid subtyping judgment. The root cause was not in the mutation or ownership machinery, but in the foundational interaction between subtyping and type-level computation: the core semantic idea that terms and types share a common language had not been studied carefully enough to support the full system built on top of it.

This paper presents *Och*, a core calculus designed to isolate and study these foundational features before reintroducing the complexity of mutation and ownership. Och combines λ -calculus with equirecursive subtyping, self-types (ι), and general recursion (*fix*) in a setting where terms and types are syntactically and semantically unified — concretely, there is no separate Π -type constructor for function types; the type of a function is itself a function, and types reduce by the same β -rule as terms. Och is the first

in a planned sequence of calculi — Och, Ochr (adding ownership/linearity), and finally Ochre (adding mutable references) — each of which answers a distinct research question while building toward the full language.

1.1 Methodology

Och has been designed experimentally: a set of *goal programs* drove the design of the typing rules. When a candidate rule set accepted all current goal programs and rejected their deliberately buggy variants, new and more demanding goal programs were added and the process iterated. This methodology has two consequences.

First, Och does not yet rest on a unifying semantic theory, and attempts at a soundness proof are ongoing. Establishing such a theory is a natural next step now that the rule set has stabilised around a substantial body of examples.

Second, the resulting test suite (see Section 5) provides strong evidence of expressivity. Using Church/Scott encodings, Och expresses booleans, natural numbers, finite sets ($\text{Fin } n$), pairs, length-indexed arrays, and safe indexing — all with dependent elimination. The type system catches bugs as subtle as adding the *wrong* two numbers (see Section 5.9), not merely checking that an addition is well-shaped.

1.2 Goals

Soundness. Ochre is only valuable if programmers can trust the compiler to catch runtime errors. Proving the soundness of a system which combines equirecursive subtyping, self-types, and general recursion in a unified term/type language is, as far as I can tell, novel. Och is the setting in which to carry out this research.

Expressivity. The type system must articulate properties and catch bugs that programmers care about. It must be expressive enough to distinguish $x + y$ from $x + x$ when one is correct and the other is not.

Extensibility. Och is the first in a series of calculi building toward Ochre. Features which do not generalise to the full language are not worth investigating here. This consideration has already manifested in the decision to use general recursion via *fix* rather than adding primitive inductive types with only well-founded induction.

2 Background

These are works which Och builds off directly, and are worth understanding on at least a surface level.

§1-3 of Hutchins (2010) is a particularly good read. It’s a 12 page compression of his 336 page thesis and is surprisingly self contained considering how novel the concepts presented are. §4-8 are not needed at all to understand Och, since they focus primarily on results and proofs.

2.1 Pure Subtype Systems

In a conventional type theory, there is a sharp syntactic distinction between terms and their types. Functions are introduced by λ and their types by Π : the term $\lambda(x : A).b$ has type $\Pi(x : A).B$. These are different constructors with different reduction rules — λ participates in β -reduction, while Π does not compute.

Pure Subtype Systems (Hutchins 2010) collapse this distinction. There is only λ : the type of a function *is* a function. Where a conventional system writes $\Pi(x : A).B$ for the type of functions from A to B , PSS writes $\lambda(x : A).B$ — the same constructor used for the functions themselves. This means types compute by exactly the same rules as terms: β -reduction, substitution, and application all apply uniformly.

The consequence is that every term is trivially its own most precise type. A value like 3 does not merely *have* type Nat ; it *is* a type, the singleton type whose only inhabitant is 3. The subtyping relation then does all the work that a typing judgment would normally do: $3 \sqsubseteq \text{Nat}$ is the statement that 3 is a natural number. There is no need for a separate $\Gamma \vdash e : \tau$ judgment — well-formedness (Section 4.2) checks structural validity, and subtyping (Section 4.4) answers every question about type compatibility.

Och adopts PSS’s core design and extends it with self-types (ι), general recursion (fix), and coinductive subtyping.

2.2 Self Types

Church encodings represent data as their own eliminators: a boolean is a function that takes two branches and returns one of them, a natural number is a function that takes a zero case and a successor case and folds over itself. This is elegant but has a well-known limitation: the eliminator’s return type cannot depend on the value being eliminated. A Church-encoded boolean can branch on itself to return values of some fixed type X , but it cannot branch to return values of type P true or P false for a type-level function P — the eliminator does not know which boolean it is.

Self types, introduced by (Fu and Stump 2014), solve this problem. The binder $\iota(x \leq \tau).e$ introduces a type in which the variable x refers to *the term inhabiting the type*. When checking whether $a \sqsubseteq \iota(x \leq \tau).e$, the rule [S-Iota-Intro] substitutes a for x in the body, reducing the goal to $a \sqsubseteq e[x \mapsto a]$. This feeds the value being typed *into* its own type, enabling the return type to depend on the inhabitant. Self types are also used by Kind (Taelin 2021a), a (now-defunct) proof language by Victor Maia (AKA Victor Taelin online), which provides an accessible introduction to the idea.

For example, the dependent boolean type `DBool` is defined so that its eliminator takes a motive $P : \text{DBool} \rightarrow \top$ and branches of type P true and P false, returning P **self** where **self** is the boolean value currently being typed. The constructors `true` and `false` are plain λ -terms, identical to their non-dependent counterparts — the dependent elimination machinery lives entirely in the type, not in the constructors. See Section 5.1 for the full definitions and Section 5 for further encodings built on this pattern.

3 Language Semantics

This section describes what each piece of the language does in natural language to give the reader an intuition for the objects involved before we get into the typing rules, examples, and metatheory. The full syntax is given in Figure 1. When the annotation τ in a binder is $\top$, we omit it: $\lambda x.e$ abbreviates $\lambda(x \leq \top).e$, and similarly for ι and fix .

$e, \tau ::=$	variable
$\lambda(x \leq \tau).e$	lambda
$e_1 e_2$	application
$\top$	universe (top)
$\perp$	primitive bottom
$\iota(x \leq \tau).e$	self-type binder
$\text{fix}(x \leq \tau).e$	recursive binder

Figure 1: Och syntax.

Variables, lambdas, application as per the standard λ -calculus, except λ has a domain type annotation which is respected by the type checker.

$\top$ **and** $\perp$ are the top and bottom of the subtyping lattice. $\top$ represents the widest possible type, which tells you nothing about the term being typed, and $\perp$ is the empty type.

Fix $\text{fix}(x \leq \tau).e$ allows a term to be defined in terms of itself. x is the reference to the whole `fix` expression itself, and τ is the “upper bound” which the whole expression can be assumed to have while it’s being type checked (which prevents loops during type checking).

Iota $\iota(x \leq \tau).e$ allows the definition of a type to include a reference to *the term inhabiting it*. This means $e \sqsubseteq \iota(x : \tau_0).\tau_1$ is reduced to $e \sqsubseteq \tau_1[x \mapsto e]$ during checking (notice: the LHS has moved to the RHS). This is a Cedille-style self type (Fu and Stump 2014), and allows for church encodings with dependent elimination instead of having to add inductive datatypes to the language as a primitive. See Section 2.2 for a longer explanation of self types.

3.1 Concrete evaluation

The concrete evaluator defines what it means for a closed term to produce a value at runtime. It plays a crucial role in the soundness statement: soundness asserts “if a program is well-formed, it will not get stuck during evaluation.” For this to be non-vacuous, the evaluator must be *partial* — it must reject ill-formed programs by having no applicable rule, rather than silently succeeding on all inputs.

Concrete evaluation is a substitution-based, call-by-value, big-step semantics on closed terms. The judgment $e \Downarrow v$ states that closed term e evaluates to value v . There is no context and no free variables: binders are eliminated by eager substitution. The rules are given in Figure 2.

The value forms are $\{\lambda, \top, \perp\}$. [E-Val] states that values evaluate to themselves. [E-App] evaluates the function to a λ , evaluates the argument, substitutes, and evaluates the body. [E-Iota] and [E-Fix] eagerly unroll by substituting the binder’s self-reference into the body and evaluating the result.

As a worked example, evaluating the application $(\lambda(x \leq \top).x) \top$:

$$(\lambda(x \leq \top).x) \top \Downarrow \top$$

by [E-App]: the function $(\lambda(x \leq \top).x)$ is already a value, the argument $\top$ is a value by [E-Val], and the substituted body $x[x \mapsto \top] = \top$ evaluates to $\top$ by [E-Val].

Aside: Och has no concept of levels or universes, so the type checker cannot enforce that $\{\top, \perp, \iota\}$ do not appear at runtime; the evaluator must handle them gracefully. Adding a universe hierarchy (see §3.6 of PSS (Hutchins 2010)) would allow type-level arguments to be erased during compilation, after which $\{\top, \perp, \iota\}$ would never appear at level 0. I plan on doing this in the future.

$$\begin{array}{c}
\frac{}{v \Downarrow v} \text{E-VAL} \qquad \frac{f \Downarrow \lambda(x \leq \tau).b \quad a \Downarrow v_a \quad b[x \mapsto v_a] \Downarrow v}{f a \Downarrow v} \text{E-APP} \\
\\
\frac{b[x \mapsto \iota(x \leq \tau).b] \Downarrow v}{\iota(x \leq \tau).b \Downarrow v} \text{E-IOTA} \qquad \frac{b[x \mapsto \text{fix}(x \leq \tau).b] \Downarrow v}{\text{fix}(x \leq \tau).b \Downarrow v} \text{E-FIX} \\
\\
\text{where } v \in \text{Value} ::= \lambda(x \leq \tau).e \quad \text{lambda} \\
\quad \quad \quad | \perp \quad \quad \text{primitive bottom} \\
\quad \quad \quad | \top \quad \quad \text{universe (top)}
\end{array}$$

Figure 2: Evaluation rules. Big-step, substitution-based, call-by-value on closed terms.

The most important aspect of the evaluation rules is what is *absent*:

- Free variables are not values and have no evaluation rule, so they cannot occur at runtime. This is what forces the type system to rule out ill-scoped variables.
- [E-App] requires the function position to evaluate to a λ . Applying $\top$, $\perp$, or any non-function term is stuck — there is no rule for it. This is what forces the type system to verify that the function position has an appropriate type before allowing an application.

A term like $\text{fix}(x \leq \top).x$ unrolls indefinitely via [E-Fix]: the body x is replaced by the whole expression, producing the same term again. This is the correct behaviour for non-terminating recursion.

4 Typing Rules

Och’s type system is structured around two judgements:

Well-formedness $\Gamma \vdash e \text{ wf}$ states “ e is well-formed under Γ ” and is the “entry point” for the type system. Defined in Section 4.2. Delegates all type-comparison questions to subtyping.

Subtyping $S; \Gamma \vdash a \sqsubseteq b$ states “ a is a subtype of b under Γ , assuming coinductive hypotheses S ”. Subtyping in Och means set inclusion: $A \sqsubseteq B$ iff every value of A is also a value of B . Section 4.3 explains the S coinductive hypothesis system.

4.1 Context Γ

Γ is the typing context defined as $\Gamma ::= \emptyset \mid \Gamma, x \leq a$. Lookup is $\Gamma(x)$. Each entry describes what the type checker knows about a variable statically.

4.2 Well-formedness

The well-formedness judgment $\Gamma \vdash e \text{ wf}$ determines whether term e is well-formed under context Γ . In other systems this role would be served by a $\Gamma \vdash a : \tau$ rule which assigns a type to a term, but there is no need to assign a type to terms in Och because every term is already trivially its own type by [S-Refl].

Well-formedness validates purely structural conditions: annotations are types, binders are in scope, and applications have Π -typed functions with domain-inhabiting arguments. The rules are given in Figure 3.

$$\begin{array}{c}
\frac{x \in \text{dom}(\Gamma)}{\Gamma \vdash x \text{ wf}} \text{W-VAR} \qquad \frac{}{\Gamma \vdash \top \text{ wf}} \text{W-TYPE} \qquad \frac{}{\Gamma \vdash \perp \text{ wf}} \text{W-BOT} \\
\\
\frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \lambda(x \leq A).b \text{ wf}} \text{W-LAM} \qquad \frac{\Gamma \vdash f \text{ wf} \quad \Gamma \vdash a \text{ wf} \quad \emptyset; \Gamma \vdash f \sqsubseteq \lambda(x \leq A).B \quad \emptyset; \Gamma \vdash a \sqsubseteq A}{\Gamma \vdash f a \text{ wf}} \text{W-APP} \\
\\
\frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \iota(x \leq A).b \text{ wf}} \text{W-IOTA} \qquad \frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \text{fix}(x \leq A).b \text{ wf}} \text{W-FIX}
\end{array}$$

Figure 3: Well-formedness rules.

[W-Var] requires variables be well-scoped.

[W-Lam], [W-Iota], [W-Fix] check that the annotation is well-formed and that the body is well-formed under the extended context.

[W-App] is the only rule with subtyping premises: the function must subtype some $\lambda(x \leq A).B$ (it has a Π -type), and the argument must subtype the domain A . No type is assigned to the result — the result’s type is the result itself (by [S-Refl]), and if the caller needs to know it subtypes something, that is a separate subtyping question.

4.3 Seen set S

Due to the prevalent usage of unbounded recursion in type definitions and encodings, many subtype checks involve infinite proof trees (e.g. $\emptyset; \emptyset \vdash \text{add one two} \sqsubseteq \text{Nat}$).

To derive these infinite trees in finite time Claude said I should use “the Brandt-Henglein device for equirecursive subtyping” as used by Brandt and Henglein (1998). Claude explains it as: “ S is a finite representation of a coinductive (potentially infinite) proof tree: when a goal recurs, the branch closes rather than recursing forever, encoding a regular infinite derivation as a finite one.”

As far as I can tell the reason it is remarkable enough to have a name is not that using it is particularly hard, but that convincing oneself it is a sound technique is hard, and Brandt/Henglein did just that. I am informed it only works when the cycles it closes contain at least one “productive” step, which are grouped together into Section 4.7.

The “real” subtyping judgment is $\emptyset; \Gamma \vdash a \sqsubseteq b$ (empty hypothesis set); non-empty S arises only inside a derivation.

4.4 Subtyping Rules

The subtyping rules fall into four categories. Each serves a distinct purpose; knowing which category a rule belongs to predicts its shape.

Category	Purpose	Rules	Extends S ?
Structural (Section 4.5)	Plumbing: recurse without inspecting constructors on either side	Figure 4	no
Congruence (Section 4.6)	Match constructor on both sides; reduce to sub-obligations with variance	Figure 5	no
Productive unfolding (Section 4.7)	Unfold a recursive binder (ι /fix); extend S ; enable coinductive closure	Figure 6	yes
Conversion (Section 4.8)	Close under head reduction so subtyping is closed under computation	Figure 7	no

Table 1: Rule taxonomy for declarative subtyping

The S -extension column matters: S-Hyp can only fire against entries that some ancestor productive rule installed, so any path to S-Hyp must cross an unfold — the productivity requirement made mechanical.

4.5 Structural rules

Structural rules (Figure 4) talk about $\sqsubseteq$ as a relation on terms without inspecting either side’s head constructor: reflexivity, transitivity, the top element, hypothesis lookup, and variable lookup.

$$\begin{array}{ccc}
\frac{}{S; \Gamma \vdash e \sqsubseteq e} \text{S-REFL} & \frac{S; \Gamma \vdash a \sqsubseteq b \quad S; \Gamma \vdash b \sqsubseteq c}{S; \Gamma \vdash a \sqsubseteq c} \text{S-TRANS} & \frac{(a, b) \in S}{S; \Gamma \vdash a \sqsubseteq b} \text{S-HYP} \\
\frac{}{S; \Gamma \vdash x \sqsubseteq \Gamma(x)} \text{S-VAR} & \frac{}{S; \Gamma \vdash e \sqsubseteq \top} \text{S-TOP} & \frac{}{S; \Gamma \vdash \perp \sqsubseteq e} \text{S-BOTL}
\end{array}$$

Figure 4: Structural subtyping rules.

Ex falso via subsumption. There is no dedicated “absurd” eliminator. If $a \sqsubseteq \perp$ is derivable, then $a \sqsubseteq e$ for every e via [S-Trans] on [S-BotL]. The “contradiction” discharge is subsumption alone. This matches the DOT (Amin et al. 2012) tradition: $\perp$ inhabits every type trivially in subtyping, so any term whose type is already $\perp$ flows into any expected type without further ceremony.

[S-Trans] is a constructor, not a derived theorem. Given the interpretation of types as sets of terms, and subtyping as subset over those sets, one would expect transitivity to be provable instead of axiomatic. Eliminating transitivity as a primitive rule (*transitivity elimination*) is highly desirable: it simplifies metatheory and makes the relation syntax-directed. PSS (Hutchins 2010) — a closely related system — did not manage to eliminate transitivity. Pasquale and García-Pérez (Pasquale and García-Pérez 2026) partially succeeded in a continuation of that work, but required switching the entire theory to a Krivine machine.

I have no satisfying answer to whether or not transitivity is eliminatable in Och, but I will try get one.

4.6 Congruence rules

Congruence rules (Figure 5) inspect the head constructor on both sides, *require it to match*, and reduce the goal to sub-obligations on the immediate sub-terms with appropriate variance.

$$\begin{array}{cc}
\frac{S; \Gamma \vdash A_2 \sqsubseteq A_1 \quad S; \Gamma, x \leq A_2 \vdash b_1 \sqsubseteq b_2}{S; \Gamma \vdash \lambda(x \leq A_1).b_1 \sqsubseteq \lambda(x \leq A_2).b_2} \text{S-LAM-CONG} & \frac{S; \Gamma \vdash f_2 \sqsubseteq f_1 \quad S; \Gamma \vdash a_2 \sqsubseteq a_1 \quad S; \Gamma \vdash a_1 \sqsubseteq a_2}{S; \Gamma \vdash f_2 \ a_2 \sqsubseteq f_1 \ a_1} \text{S-APP-CONG} \\
\frac{S; \Gamma \vdash A_1 \sqsubseteq A_2 \quad S; \Gamma, x \leq A_2 \vdash B_1 \sqsubseteq B_2}{S; \Gamma \vdash \iota(x \leq A_1).B_1 \sqsubseteq \iota(x \leq A_2).B_2} \text{S-IOTA-CONG} & \frac{S; \Gamma \vdash A_1 \sqsubseteq A_2 \quad S; \Gamma, x \leq A_2 \vdash b_1 \sqsubseteq b_2}{S; \Gamma \vdash \text{fix}(x \leq A_1).b_1 \sqsubseteq \text{fix}(x \leq A_2).b_2} \text{S-FIX-CONG}
\end{array}$$

Figure 5: Congruence subtyping rules.

The equivalence premise on [S-App-Cong] (both directions on the argument) is necessary because a neutral head can use its argument at any variance, so equivalence is the only sound congruence.

4.7 Productive unfolding (extend S)

A rule is *productive* when it replaces a goal whose head is a recursive binder (ι or fix) with a goal where that binder has been unfolded once. Unfolding makes the term *larger* in syntactic size, so no structural induction can traverse an arbitrary chain of unfolds. Productivity instead provides a *coinductive* handle: once at least one unfold has fired, the original goal is guaranteed to eventually re-appear as an ancestor, at which point [S-Hyp] can close the derivation.

The rules (Figure 6) are the only ones that extend S .

$$\begin{array}{c}
\frac{S'; \Gamma \vdash a \sqsubseteq A}{S'; \Gamma \vdash a \sqsubseteq B[x \mapsto a]} \text{S-IOTA-INTRO} \quad \frac{S'; \Gamma \vdash B[x \mapsto \iota(x \leq A).B] \sqsubseteq c}{S; \Gamma \vdash \iota(x \leq A).B \sqsubseteq c} \text{S-UNFOLD-IOTA-L} \\
\\
\frac{S'; \Gamma \vdash a \sqsubseteq B[x \mapsto \iota(x \leq A).B]}{S; \Gamma \vdash a \sqsubseteq \iota(x \leq A).B} \text{S-UNFOLD-IOTA-R} \\
\\
\frac{S'; \Gamma \vdash b[x \mapsto \text{fix}(x \leq A).b] \sqsubseteq c}{S; \Gamma \vdash \text{fix}(x \leq A).b \sqsubseteq c} \text{S-UNFOLD-FIX-L} \quad \frac{S'; \Gamma \vdash a \sqsubseteq b[x \mapsto \text{fix}(x \leq A).b]}{S; \Gamma \vdash a \sqsubseteq \text{fix}(x \leq A).b} \text{S-UNFOLD-FIX-R}
\end{array}$$

Figure 6: Productive unfolding rules. $S' = S, (a, b)$ where $S; \Gamma \vdash a \sqsubseteq b$ is the current goal.

S-Unfold-Iota-R is the weaker sibling of S-Iota-Intro (same conclusion, no annotation premise), needed to close the equivalence of ι -unfolding.

S-Iota-Intro is **the** rule which allows dependent elimination: it puts the thing being typed **into** the type, allowing the type to depend on the inhabitant.

Worked S-Iota-Intro example: $\text{true} \sqsubseteq \text{DBool}$. With

$$\begin{aligned}
\text{true} &:= \lambda(P \leq \top). \lambda(t \leq \top). \lambda(f \leq \top). t \\
\text{false} &:= \lambda(P \leq \top). \lambda(t \leq \top). \lambda(f \leq \top). f \\
\text{DBool} &:= \text{fix}(B \leq \top). \iota(\text{self} \leq B). \lambda(P \leq B \rightarrow \top). \lambda(t \leq P \text{ true}). \lambda(f \leq P \text{ false}). P \text{ self}
\end{aligned}$$

the constructors are plain lambdas with $\top$ domains — they carry no recursive reference to DBool . The subtyping check $\emptyset; \Gamma \vdash \text{true} \sqsubseteq \text{DBool}$ still requires the coinductive seen-set to close, because [S-Iota-Intro]’s annotation premise cycles back to the root goal:

$$\begin{array}{ll}
\emptyset; \Gamma \vdash \text{true} \sqsubseteq \text{DBool} & \text{root goal} \\
S_1; \Gamma \vdash \text{true} \sqsubseteq \iota(\text{self} \leq \text{DBool}). \lambda(P \leq \text{DBool} \rightarrow \top). \lambda(t \leq & \text{S-Unfold-Fix-R} \\
P \text{ true}). \lambda(f \leq P \text{ false}). P \text{ self} & \\
\text{where } S_1 = \{(\text{true}, \text{DBool})\} & \\
\text{S-Iota-Intro — annotation premise:} & \\
S_1; \Gamma \vdash \text{true} \sqsubseteq \text{DBool} & \text{S-Hyp } \checkmark \\
\text{root goal reappears — closed by } (\text{true}, \text{DBool}) \in S_1 & \\
\text{S-Iota-Intro — body premise (after } [\text{self} \mapsto \text{true}]): & \\
S_1; \Gamma \vdash \text{true} \sqsubseteq \lambda(P \leq \text{DBool} \rightarrow \top). \lambda(t \leq P \text{ true}). \lambda(f \leq & \text{S-Lam} \\
P \text{ false}). P \text{ true} & \\
\text{contravariant: } (\text{DBool} \rightarrow \top) \sqsubseteq \top \text{ by S-Top } \checkmark & \\
\text{covariant body under } P \leq \text{DBool} \rightarrow \top: & \\
S_1; \Gamma \vdash \lambda(t \leq \top). \lambda(f \leq \top). t \sqsubseteq \lambda(t \leq & \text{S-Lam} \\
P \text{ true}). \lambda(f \leq P \text{ false}). P \text{ true} & \\
\text{contravariant: } P \text{ true} \sqsubseteq \top \text{ by S-Top } \checkmark & \\
\text{covariant body under } t \leq P \text{ true: S-Lam again} & \\
\text{contravariant: } P \text{ false} \sqsubseteq \top \text{ by S-Top } \checkmark & \\
\text{covariant body: } t \sqsubseteq P \text{ true by S-Var } (t \leq P \text{ true} \in \Gamma) \checkmark &
\end{array}$$

Because the constructors have $\top$ domains, every contravariant [S-Lam] premise is discharged by [S-Top]. The only coinductive step is the [S-Iota-Intro] annotation premise, which closes via [S-Hyp] after one productive unfold. This is the Brandt–Henglein discipline in action: recursion in the subtyping judgment is legal *only across a productive unfold*, so non-productive loops (e.g. reflexivity-by-loop) cannot sneak through.

4.8 Conversion

The subtyping relation must be closed under head reduction: if a computes to a' , then $a \sqsubseteq b$ should hold iff $a' \sqsubseteq b$. The conversion rules (Figure 7) provide that closure.

$$\frac{S; \Gamma \vdash \text{body}[x \mapsto \arg] \sqsubseteq b}{S; \Gamma \vdash (\lambda(x \leq A). \text{body}) \arg \sqsubseteq b} \text{S-BETA-L} \qquad \frac{S; \Gamma \vdash a \sqsubseteq \text{body}[x \mapsto \arg]}{S; \Gamma \vdash a \sqsubseteq (\lambda(x \leq A). \text{body}) \arg} \text{S-BETA-R}$$

Figure 7: Conversion rules.

5 Example Programs

All encodings below are Church/Scott-style: data types are represented as their own eliminators, with no primitive inductive types in the language. Every definition is mechanised and tested in Lean 4 (see Section 7). This section builds from simple encodings to two programs that demonstrate Och’s ability to catch subtle bugs in dependently-typed code. Throughout, underlined names (true, Nat, etc.) denote meta-level definitions, as opposed to object-level bound variables (x , n , P , etc.).

5.1 Booleans

$\begin{aligned} \underline{\text{true}} &:= \lambda_. \lambda t. \lambda_. t \\ \underline{\text{false}} &:= \lambda_. \lambda_. \lambda f. f \\ \underline{\text{Bool}} &:= \lambda X. \lambda(t \leq X). \lambda(f \leq X). X \end{aligned}$	<pre>data Bool : Set where true : Bool false : Bool</pre> (roughly equivalent Agda)
---	---

Bool is a standard Church-encoded boolean: the eliminator takes two branches and a return type X , and the type itself is X . Elimination is non-dependent — the return type X is the same regardless of whether the value is true or false.

$\begin{aligned} \underline{\text{DBool}} &:= \text{fix } B. \iota(\text{self} \leq B). \\ &\lambda(P \leq B \rightarrow \top). \lambda(t \leq P \underline{\text{true}}). \lambda(f \leq P \underline{\text{false}}). P \text{ self} \end{aligned}$	<pre>data DBool : Set where true : DBool false : DBool</pre> (roughly equivalent Agda)
--	--

DBool is the *dependent* counterpart: its motive P is a function from values to types, so the return type $P \text{ self}$ varies with the value being eliminated. This is what ι enables — the self binder in $\iota(\text{self} \leq B)$ refers to the actual value inhabiting the type, and [S-Iota-Intro] substitutes it into $P \text{ self}$ during type checking.

Crucially, Bool and DBool *share the same constructors* $\{\underline{\text{true}}, \underline{\text{false}}\}$ (Figure 8). The dependent elimination machinery lives entirely in the type (DBool’s ι and motive), not in the constructors. At runtime, a DBool value is identical to a Bool value — the difference is purely compile-time. This pattern recurs later with Fin reusing Nat’s constructors (Section 5.4).

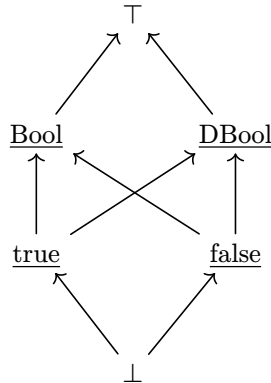


Figure 8: Subtyping lattice for booleans. Arrows denote $\sqsubseteq$.

5.2 Pairs

$\underline{\text{Pair}} := \lambda A. \lambda B. \lambda(X \leq \top). \lambda(k \leq A \rightarrow B \rightarrow X). X$ $\underline{\text{pair}} := \lambda A. \lambda B. \lambda(a \leq A). \lambda(b \leq B). \lambda(X \leq \top). \lambda(k \leq A \rightarrow B \rightarrow X). kab$	<pre>record Pair (A B : Set) : Set where constructor pair field fst : A; snd : B</pre> (roughly equivalent Agda)
---	---

Church-encoded binary products. Pair projections fst/snd erase the unused component's type to $\top$, so call sites only supply the type of the component being projected.

5.3 Natural numbers

$\underline{\text{zero}} := \lambda(P \leq \top). \lambda(z \leq \top). \lambda(s \leq \top). z$ $\underline{\text{succ}} := \lambda(n \leq \top). \lambda(P \leq \top). \lambda(z \leq \top). \lambda(s \leq n \rightarrow \top). s \ n$ $\underline{\text{Nat}} := \text{fix}(N \leq \top). \iota(\text{self} \leq N). \lambda(P \leq N \rightarrow \top). \lambda(z \leq P \ \underline{\text{zero}}). \lambda(s \leq \lambda(n \leq N). P \ (\underline{\text{succ}} \ n)). P \ \text{self}$	<pre>data Nat : Set where zero : Nat succ : Nat → Nat</pre> (roughly equivalent Agda)
--	--

Nat uses both key features of Och. The outer fix ties the recursive knot (Nat refers to itself in the motive domain). The inner ι binds self to the value being typed, enabling *dependent* elimination: the return type $P \ \text{self}$ varies with the value.

The constructors are plain lambdas with $\top$ domains — they carry no reference to Nat, so $\underline{\text{zero}} \sqsubseteq \underline{\text{Nat}}$ and $\underline{\text{succ}} \ n \sqsubseteq \underline{\text{Nat}}$ hold by the coinductive seen-set discipline (productive unfold of fix/ ι , then all contravariant domain checks discharge via [S-Top]).

Addition is defined with an explicit fix since the Scott-style eliminator provides no induction hypothesis:

$\underline{\text{add}} := \text{fix}(\text{add} \leq \underline{\text{Nat}} \rightarrow \underline{\text{Nat}} \rightarrow \underline{\text{Nat}}). \lambda(n \leq \underline{\text{Nat}}). \lambda(m \leq \underline{\text{Nat}}). n \ (\underline{\text{Nat}} \rightarrow \underline{\text{Nat}}) \ m \ (\lambda(\text{pred} \leq \underline{\text{Nat}}). \underline{\text{succ}} \ (\text{add} \ \text{pred} \ m))$	<pre>add : Nat → Nat → Nat add zero m = m add (succ pred) m = succ (add pred m)</pre> (roughly equivalent Agda)
--	---

5.4 Finite sets

Fin n is the type of naturals strictly less than n . The encoding strategy is: eliminate n to compute the type.

- The zero case yields $\perp$ (uninhabited — there are no naturals less than zero).
- The successor case yields an ι -type which is the same as Nat except it is more precise about which number is passed to the succ case fs.

The overall structure — case-split on the index, $\perp$ for the empty case, ι for the successor case — is shared with Kind (Taelin 2021b), which uses self-types nearly identically to ι (see base/Fin.kind in the Kind-Legacy repository). The general machinery for elaborating inductive definitions to lambda-encodings with self-types is developed by (Fu and Stump 2014) and extended to indexed families in Cedille's implementation, though no published work gives an explicit lambda-encoding of Fin specifically.

```

Fin := fix( $F \leq \underline{\text{Nat}} \rightarrow \top$ ). $\lambda(n \leq \underline{\text{Nat}})$ .
   $n (\lambda(\_ \leq \underline{\text{Nat}}).\top) \perp$ 
  ( $\lambda(\text{pred} \leq \underline{\text{Nat}})$ .
     $\iota(\text{self} \leq \underline{\text{Nat}})$ .
     $\lambda(P \leq \underline{\text{Nat}} \rightarrow \top)$ .
     $\lambda(\text{fz} \leq P \text{ zero})$ .
     $\lambda(\text{fs} \leq \lambda(q \leq F \text{ pred})$ .
       $P (\text{succ } q)$ .
     $P \text{ self})$ 

```

```

data Fin : Nat → Set where
  fzero : Fin (succ n)
  fsucc  : Fin n → Fin (succ n)
(roughly equivalent Agda)

```

A key design choice is that `self` is typed as a `Nat`, not as a `Fin`. This means every `Fin` value is automatically a `Nat` value ($\underline{\text{Fin}}\ n \sqsubseteq \underline{\text{Nat}}$), which is the correct set-theoretic relationship. Kind’s encoding (Taelin 2021b) types `self` as `Fin(Nat.succ(lim.pred))` instead, which requires an explicit coercion function `Fin.to_nat` to recover the embedding.

Natural number literals inhabit `Fin` by subsumption: $\text{zero} \sqsubseteq \underline{\text{Fin}} (\text{succ } n)$ holds for any n , and $n \sqsubseteq \underline{\text{Fin}}\ n$ is correctly rejected (the diagonal). No separate `FZ/FS` constructors are needed — the `Nat` constructors are reused directly, just as `Bool` and `DBool` share constructors (Section 5.1). Figure 9 illustrates the subtyping relationships.

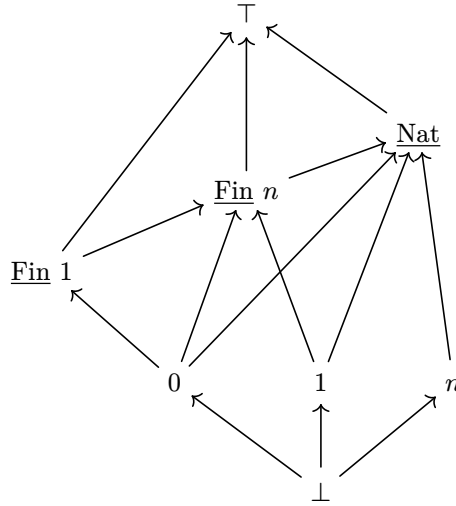


Figure 9: Subtyping lattice for natural numbers and finite sets. Laid out such that subtypes are physically beneath their supertypes.

5.5 Length-indexed arrays

```

Array := fix(Arr ≤ Nat → Top → Top).
   $\lambda(n \leq \underline{\text{Nat}}).\lambda(T \leq \top)$ .
   $n (\lambda(\_ \leq \underline{\text{Nat}}).\top)$ 
  Unit
  ( $\lambda(\text{pred} \leq \underline{\text{Nat}})$ .
    Pair  $T (\text{Arr pred } T)$ )

```

```

Array : Nat → Set → Set
Array zero    T = Unit
Array (succ n) T = Pair T (Array n T)
(roughly equivalent Agda)

```

$\underline{\text{Array}}\ n\ T$ computes by eliminating the length index: $\underline{\text{Array}}\ \text{zero}\ T \equiv \underline{\text{Unit}}$ and $\underline{\text{Array}}\ (\text{succ } k)\ T \equiv \underline{\text{Pair}}\ T\ (\underline{\text{Array}}\ k\ T)$. Arrays are built with `unit/pair` directly.

5.6 Dependent pairs

The dependent pair `Sigma` packages a witness with a payload whose type depends on the witness. This is a primitive in Agda (Σ); in Och it is Church-encoded:

$$\begin{aligned}
\underline{\text{Sigma}} &:= \lambda A. \lambda (B \leq A \rightarrow \top). \\
&\quad \lambda (X \leq \top). \lambda (k \leq \lambda (a \leq A). B \ a \rightarrow X). X \\
\underline{\text{dpair}} &:= \lambda A. \lambda (B \leq A \rightarrow \top). \lambda (a \leq A). \lambda (b \leq B \ a). \\
&\quad \lambda (X \leq \top). \lambda (k \leq \lambda (a \leq A). B \ a \rightarrow X). k \ a \ b
\end{aligned}$$

5.7 Vectors

Vectors package a length with an array of that length:

$$\begin{aligned}
\underline{\text{Vec}} \ T &:= \underline{\text{Sigma}} \ \underline{\text{Nat}} \ (\lambda (n \leq \underline{\text{Nat}}). \underline{\text{Array}} \ n \ T) & \underline{\text{Vec}} \ : \ \text{Set} \rightarrow \text{Set} \\
& & \underline{\text{Vec}} \ T &= \Sigma \ \underline{\text{Nat}} \ (\lambda \ n \rightarrow \underline{\text{Array}} \ n \ T) \\
& & & \text{(roughly equivalent Agda)}
\end{aligned}$$

5.8 appendArrays

appendArrays concatenates two arrays by recursion on the first array's length index:

$$\begin{aligned}
\underline{\text{appendArrays}} &:= & \underline{\text{appendArrays}} &: \forall \{T \ n_1 \ n_2\} \\
& \text{fix}(\text{self} \leq \underline{\text{Nat}} \rightarrow \underline{\text{Nat}} \rightarrow \top \rightarrow \top). & & \rightarrow \underline{\text{Array}} \ n_1 \ T \rightarrow \underline{\text{Array}} \ n_2 \ T \\
& \lambda (T \leq \top). \lambda (n_1 \leq \underline{\text{Nat}}). \lambda (n_2 \leq \underline{\text{Nat}}). & & \rightarrow \underline{\text{Array}} \ (\text{add } n_1 \ n_2) \ T \\
& \lambda (\text{arr}_1 \leq \underline{\text{Array}} \ n_1 \ T). & \underline{\text{appendArrays}} \ \{n_1 = \text{zero}\} \ _ \ \text{arr}_2 &= \text{arr}_2 \\
& \lambda (\text{arr}_2 \leq \underline{\text{Array}} \ n_2 \ T). & \underline{\text{appendArrays}} \ \{n_1 = \text{succ } p\} \ \text{arr}_1 \ \text{arr}_2 &= \\
& n_1 & \text{let } (\text{hd} \ , \ \text{tl}) = \text{arr}_1 \text{ in} & \\
& & \text{(hd} \ , \ \underline{\text{appendArrays}} \ \text{tl} \ \text{arr}_2) & \\
& & \text{(roughly equivalent Agda)} & \\
& (\lambda (n \leq \underline{\text{Nat}}). \underline{\text{Array}} \ n \ T) & & \\
& \rightarrow \underline{\text{Array}} \ (\text{add } n \ n_2) \ T) & & \\
& (\lambda (_ \leq \underline{\text{Array}} \ \text{zero} \ T). \text{arr}_2) & & \\
& (\lambda (\text{pred} \leq \underline{\text{Nat}}). & & \\
& \lambda (\text{arr} \leq \underline{\text{Array}} \ (\text{succ} \ \text{pred}) \ T). & & \\
& \underline{\text{pair}} \dots (\text{fst} \ \text{arr}) & & \\
& \quad (\text{self } T \ \text{pred} \ n_2 & & \\
& \quad \quad (\text{snd} \ \text{arr}) \ \text{arr}_2)) & & \\
& \text{arr}_1 & &
\end{aligned}$$

The zero case returns the second array unchanged. The successor case deconstructs the first array as a pair, keeps the head, and recursively appends the tail.

5.9 appendVec: catching the wrong addition

The north-star example. appendVec unpacks two vectors, concatenates their arrays with appendArrays, and repacks the result with the summed length using dpair:

appendVec := $\lambda(T \leq \top).$

$\lambda(v_1 \leq \text{Vec } T). \lambda(v_2 \leq \text{Vec } T).$

$v_1 (\text{Vec } T)$

$(\lambda(n_1 \leq \text{Nat}). \lambda(\text{arr}_1 \leq \text{Array } n_1 T).$

$v_2 (\text{Vec } T)$

$(\lambda(n_2 \leq \text{Nat}). \lambda(\text{arr}_2 \leq \text{Array } n_2 T).$

$\text{dpair Nat } \dots$

$(\text{add } n_1 n_2)$

$(\text{appendArrays } T n_1 n_2 \text{ arr}_1 \text{ arr}_2)))$

appendVec : $\forall \{T\} \rightarrow \text{Vec } T \rightarrow \text{Vec } T \rightarrow \text{Vec } T$

appendVec (n_1 , arr_1) (n_2 , arr_2) =
($\text{add } n_1 n_2$, $\text{appendArrays } \text{arr}_1 \text{ arr}_2$)

(roughly equivalent Agda)

This type-checks: the length add $n_1 n_2$ matches the length of the concatenated array.

Now consider a version with a deliberate bug — add $n_1 n_1$ instead of add $n_1 n_2$:

appendVec_{wrong} := $\dots \text{dpair Nat } \dots (\text{add } n_1 n_1) (\text{appendArrays } T n_1 n_2 \text{ arr}_1 \text{ arr}_2) \dots$

The type checker *rejects* this: $\text{arr}_2 \sqsubseteq \text{Array } n_1 T$ fails because arr_2 has type $\text{Array } n_2 T$ and $n_2 \neq n_1$. The system catches a bug as subtle as adding the wrong two numbers.

5.10 indexArr: safe array indexing

indexArr looks up the i -th element of an Array $n T$, where i has type Fin n :

indexArr := fix self.

$\lambda(T \leq \top). \lambda(n \leq \text{Nat}).$

$n (\lambda(m \leq \text{Nat}).$

$\text{Array } m T \rightarrow \text{Fin } m \rightarrow T)$

$(\lambda(\text{arr} \leq \text{Array } \text{zero } T).$

$\lambda(i \leq \text{Fin } \text{zero}). i)$

$(\lambda(p \leq \text{Nat}).$

$\lambda(\text{arr} \leq \text{Array } (\text{succ } p) T).$

$\lambda(i \leq \text{Fin } (\text{succ } p)). \dots)$

indexArr : $\forall \{T\} n$

$\rightarrow \text{Array } n T \rightarrow \text{Fin } n \rightarrow T$

indexArr { $n = \text{zero}$ } $\text{arr } i = \text{absurd } i$

indexArr { $n = \text{succ } p$ } $\text{arr } i =$

let (hd , tl) = arr in

$i (\lambda _ \rightarrow T)$

hd

$(\lambda j \rightarrow \text{indexArr } \text{tl } j)$

(roughly equivalent Agda)

In the zero-length branch, i has type Fin zero = $\perp$, so the branch body is i itself — ex falso via [S-BotL]. In the successor branch, the array is deconstructed as a pair and i eliminates to either the head or a recursive call on the tail.

The payoff is at call sites:

indexArr Nat three $\text{arr } \text{zero}$ ✓ (zero $\sqsubseteq$ Fin three)

indexArr Nat three $\text{arr } \text{two}$ ✓ (two $\sqsubseteq$ Fin three)

indexArr Nat three $\text{arr } \text{three}$ × (three $\not\sqsubseteq$ Fin three)

Out-of-bounds access is a *compile-time* error: the index literal fails to subtype Fin n , and no runtime check is needed.

5.11 Eater: concrete evaluation with fix

The following example demonstrates concrete evaluation of a recursive term defined via fix.

$$\begin{aligned}\underline{\text{bool}} &:= \lambda t. \lambda f. \top \\ \underline{\text{true}} &:= \lambda t. \lambda f. t \\ \underline{\text{false}} &:= \lambda t. \lambda f. f \\ \underline{\text{eater}} &:= \text{fix } e. \lambda (b \leq \underline{\text{bool}}). b \top e\end{aligned}$$

$\underline{\text{eater}}$ is a recursive function that consumes $\underline{\text{false}}$ arguments until it hits a $\underline{\text{true}}$, at which point it returns $\top$. At each step, fix unrolls via [E-Fix], and the boolean argument selects between $\top$ (stop) and the self-reference e (continue). E.g. $\underline{\text{eater}} \ \underline{\text{true}} \Downarrow \top$ and similarly $\underline{\text{eater}} \ \underline{\text{false}} \ \underline{\text{false}} \ \underline{\text{false}} \ \underline{\text{true}} \Downarrow \top$.

Evaluation of $\underline{\text{eater}} \ \underline{\text{false}} \ \underline{\text{true}}$. We abbreviate $\underline{\text{eater}}$'s unrolled form $\lambda (b \leq \underline{\text{bool}}). b \top \underline{\text{eater}}$ as $\underline{\text{eater}}^\circ$ for readability.

$\underline{\text{eater}} \ \underline{\text{false}} \ \underline{\text{true}} \Downarrow \top$	E-App
$\underline{\text{eater}} \ \underline{\text{false}} \Downarrow \underline{\text{eater}}^\circ$	E-App
$\underline{\text{eater}} \Downarrow \underline{\text{eater}}^\circ$	E-Fix
$\underline{\text{false}} \Downarrow \underline{\text{false}}$	E-Val
$(b \top \underline{\text{eater}})[b \mapsto \underline{\text{false}}] \Downarrow \underline{\text{eater}}^\circ$	$\underline{\text{false}}$ selects second arg
$\underline{\text{true}} \Downarrow \underline{\text{true}}$	E-Val
$(b \top \underline{\text{eater}})[b \mapsto \underline{\text{true}}] \Downarrow \top$	$\underline{\text{true}}$ selects first arg ✓

Each [E-Fix] step replaces the bound variable e with the entire fix expression, producing a λ that is ready to accept the next argument. The recursion terminates when $\underline{\text{true}}$ selects $\top$ instead of the self-reference.

Evaluation of $\underline{\text{eater}} \ \underline{\text{true}} \ \underline{\text{false}}$ (explodes):

$\underline{\text{eater}} \ \underline{\text{true}} \ \underline{\text{false}} \Downarrow ?$	E-App
$\underline{\text{eater}} \ \underline{\text{true}} \Downarrow \top$	as above — $\underline{\text{true}}$ selects first arg
$\underline{\text{false}} \Downarrow \underline{\text{false}}$	E-Val
$\top \ \underline{\text{false}} \Downarrow ?$	stuck — $\top$ is not a $\lambda \times$

There is no rule whose conclusion matches $\top \ \underline{\text{false}} \Downarrow v$: [E-App] requires the function position to evaluate to a λ , and $\top$ is not one. The derivation is stuck — this is exactly the kind of runtime error the type system is designed to prevent.

6 Metatheory

Soundness connects the concrete semantics (Section 3.1), subtyping (Section 4.4), and well-formedness (Section 4.2).

Evaluation equivalence. If e evaluates to e' , both directions of subtyping hold.

$$e \Downarrow e' \text{ implies } \emptyset; \emptyset \vdash e' \sqsubseteq e \text{ and } \emptyset; \emptyset \vdash e \sqsubseteq e'$$

Progress. A well-formed closed term doesn't get stuck during evaluation. The evaluator may return a value or run out of fuel but never reaches an error state.

End-to-end. Composing the above: if e is well-formed and subtypes τ , and e evaluates to e' , the result subtypes τ .

$$\emptyset \vdash e \text{ wf and } \emptyset; \emptyset \vdash e \sqsubseteq \tau \text{ and } e \Downarrow e' \text{ implies } \emptyset; \emptyset \vdash e' \sqsubseteq \tau$$

6.1 What soundness promises to the programmer

Two separable runtime properties a type system can promise:

1. **Termination.** Running a well-typed program eventually produces a value. A totality / normalization claim.
2. **No runtime type errors.** Conditional on the program *not* diverging, the value it produces is well-formed and the program never reaches a stuck state.

Och **does not aim to prove (1)**. Consistency and normalization are explicitly deferred; the calculus admits non-terminating terms by design (type-in-type; general recursion via fix).

Och **does aim to prove (2)**, and this is the real runtime guarantee of the type system. The language expects the following discipline: **type-check first; only evaluate if it succeeded**.

7 Lean Formalisation

The calculus described above has been formalised in Lean 4, which is used to maintain a test suite of example programs and allow AI agents to iterate on the metatheory. The source is available at github.com/charlielidbury/ochre.

8 Related Work

Large-scale verification projects have demonstrated that formal verification of systems software is feasible: CompCert (Leroy 2009) is a verified C compiler (written in Coq, extracted to OCaml), seL4 (Klein et al. 2009) is a verified OS microkernel (C code with a separate Isabelle/HOL refinement proof), and Project Everest (Bhargavan et al. 2017) produced a verified HTTPS stack (F*, extracted to C via KReMLin). These projects validate the *goal* of verified systems software but do not provide a programming model where the systems programmer writes verified code directly — the verification happens in a separate language from the executable artifact.

The approaches below *do* aim to give the programmer a direct path from code to proof. To make the comparison concrete, each subsection illustrates how safe array indexing — accessing element i of an array of length n with a static guarantee that $i < n$, eliminating the runtime bounds check — is expressed under each approach.

8.1 Verification via extraction

In the extraction approach, the programmer writes verified code in a theorem prover and a compiler extracts executable systems code. Low* (the systems fragment of F* (Protzenko et al. 2017)) is the most developed example: after verification, KReMLin extracts Low* to idiomatic C. Safe indexing into a length-indexed vector is expressed via a refinement type on the index:

```
type vec (a:Type) : nat -> Type =
  | Nil : vec a 0
  | Cons : #n:nat -> hd:a -> tl:vec a n -> vec a (n + 1)

let rec get #a #n (i:nat{i < n}) (v:vec a n) : a =
  let Cons hd tl = v in
  if i = 0 then hd else get (i - 1) tl
```

Listing 1: Safe array indexing in F*. The refinement $i:\text{nat}\{i < n\}$ is a compile-time proof obligation; callers must prove $i < n$ to the type checker. The `Nil` case is statically unreachable.

For systems code, Low*’s buffer operations carry the same refinement pattern: `b.(n)` requires a proof that $n < \text{length } b$, and after verification KReMLin extracts this to a bare C array access `b[n]` with no bounds check. The result is zero-overhead verified code, but the programmer must express their algorithm in F*’s purely functional, monadic-effect style, thread heap invariants through a state monad, and maintain separation-logic-like permissions on buffer liveness — all of which are far removed from the C code they ultimately target. The development cost is very high: the ratio of proof to code is typically 10:1 or greater.

8.2 Translation-based verification

Aeneas (Ho and Protzenko 2022) takes an existing Rust program, translates it to a pure functional model in a theorem prover (Lean, Coq, F*, or HOL4), and lets the programmer verify properties of the model. The key insight is that Rust’s ownership discipline guarantees that each mutable borrow has exclusive access, so an in-place mutation `x[i] = v` can be modelled as a pure functional update that returns a new collection.

```

// Rust source
pub fn zero(x: &mut Vec<u32>) {
  let mut i = 0;
  while i < x.len() {
    x[i] = 0;
    i += 1;
  }
}

-- Aeneas-generated Lean
def zero_loop (x : Vec U32) (i : Usize)
  : Result (Vec U32) :=
  if i < Vec.len x then do
    let (_, back) ←
      Vec.index_mut x i
    let x1 := back 0#u32
    zero_loop x1 (i + 1#usize)
  else Result.ok x

```

Listing 2: Aeneas translation of in-place vector zeroing. The `&mut Vec<u32>` becomes a value that is returned; `x[i] = 0` becomes a call to `index_mut` returning a backward continuation `back` which, applied to the new value, produces the updated vector. Failable operations (indexing, arithmetic) live inside the `Result` monad.

This avoids the need to write systems code in a theorem prover, but introduces an architectural boundary: the programmer writes Rust, the toolchain generates functional models, and the prover reasons about those models. The verified artifact and the executable artifact are different objects in different languages, so proof obligations live outside the source language and cannot influence compilation (e.g., by eliding a bounds check). The programmer must also trust the translation and the Rust compiler.

8.3 Bolt-on verification for Rust

Prusti (Astrauskas et al. 2022), Creusot (Denis et al. 2022), and Verus (Lattuada et al. 2023) add specification and verification capabilities to Rust via annotations, macros, or embedded DSLs. The programmer writes Rust code as normal and adds pre/postconditions and loop invariants; an automated verifier (typically backed by an SMT solver) checks them.

```

// Verus: verified binary search (from verus-lang/verus examples)
verus! {
  fn binary_search(v: &Vec<u64>, k: u64) -> (r: usize)
    requires
      forall|i: int, j: int|
        0 <= i <= j < v.len() ==> v[i] <= v[j],
      exists|i: int| 0 <= i < v.len() && k == v[i],
    ensures
      r < v.len(),
      k == v[r as int],
  {
    let mut lo: usize = 0;
    let mut hi: usize = v.len() - 1;
    while lo != hi
      invariant
        hi < v.len(),
        exists|i: int| lo <= i <= hi && k == v[i],
        forall|i: int, j: int|
          0 <= i <= j < v.len() ==> v[i] <= v[j],
        decreases hi - lo,
      {
        let mid = lo + (hi - lo) / 2;
        if v[mid] < k { lo = mid + 1; } else { hi = mid; }
      }
    lo
  }
} // verus!

```

Listing 3: Verified binary search in Verus. Specifications are written in a first-order logic DSL inside `requires/ensures/invariant` blocks. Each `v[mid]` access is verified to be in bounds from the loop invariant. Specifications operate over a ghost `Seq<T>` model, not Rust’s native `Vec<T>`.

These tools have the lowest adoption barrier: the programmer stays in Rust and adds specifications incrementally. However, the specification and implementation remain in separate languages that the programmer must keep in sync. Specifications operate over ghost models (`Seq<T>`, accessed via `v@`) since

Rust’s native types cannot appear in logical formulas. The expressivity of specifications is limited by what the SMT backend can decide, and the verification is incomplete — the solver may time out or fail to find a proof, requiring manual intervention in the form of assertions or lemma calls. In a language with native dependent types, by contrast, the precondition $i < n$ is the type of the index argument, not a side-channel annotation, and the type checker itself is the verification engine.

8.4 Dependently typed systems languages

ATS (Xi 2017) and Low* (the systems fragment of F* (Protzenko et al. 2017)) are the closest prior art to Ochre’s vision: languages which combine dependent types with low-level control in a single system.

```
(* ATS: safe array access via dependent sorts *)
fun{a:t@ype} arrayref_get_at
  {n:int}{i:nat | i < n}
  (A: arrayref(a, n), i: size_t i): (a)

(* Binary search with compile-time bounds proof *)
fun{a:t@ype} bsearch_arr{n:nat}
  (A: arrayref(a, n), n: int n, x0: a,
   cmp: (a, a) -> int) : int = let
  fun loop {i,j:int | 0 <= i; i <= j+1; j+1 <= n}
    (A: arrayref(a, n), l: int i, u: int j)
    :<clorefl> int =
    if l <= u then let
      val m = l + half(u - l)
      val x = A[m] // bounds proof discharged by {i,j} constraint
      val sgn = cmp(x0, x)
    in
      if sgn >= 0 then loop(A, m+1, u) else loop(A, l, m-1)
    end else u
  in loop(A, 0, n-1) end
```

Listing 4: Safe array access in ATS. The constraint $\{i,j:int \mid 0 \leq i; i \leq j+1; j+1 \leq n\}$ on the inner loop statically guarantees every $A[m]$ access is in bounds. ATS’s constraint solver discharges these obligations automatically.

ATS demonstrates that statically guaranteed bounds safety in systems code is achievable, but at a significant ergonomic cost: programmers must work within a bespoke constraint language (the $\{\dots\}$ sort annotations) that is syntactically and semantically separated from the term language. Low* achieves similar guarantees via refinement types and SMT discharge, but inherits F*’s purely functional programming model — mutation is expressed via a monadic state effect, not via direct mutable references as in Rust.

Ochre aims to differ from both by adopting Rust’s ownership model as the mechanism for safe mutation, rather than linear types (ATS) or monadic effects (Low*), and by unifying the type and term languages so that specifications are written in the same language as the code they describe. The hypothesis is that ownership is a more natural fit for systems programmers, since Rust has demonstrated that the model is learnable at scale, and that a unified language reduces the cognitive overhead of switching between specification and implementation.

Bibliography

- Amin, Nada, Adriaan Moors, and Martin Odersky. 2012. “Dependent Object Types.” In “Workshop on Foundations of Object-Oriented Languages (Fool).” Special issue, *Workshop on Foundations of Object-Oriented Languages (FOOL)* (Tucson, AZ, USA),. <https://infoscience.epfl.ch/server/api/core/bitstreams/678b1818-06aa-47f1-b7e0-783413d76044/content>.
- Astrauskas, Vytautas, Aurel Bily, Jonás Fiala, et al. 2022. “The Prusti Project: Formal Verification for Rust.” In “NASA Formal Methods (Nfm).” Special issue, *NASA Formal Methods (NFM)*,. https://doi.org/10.1007/978-3-031-06773-0_5.

- Bhargavan, Karthikeyan, Barry Bond, Antoine Delignat-Lavaud, et al. 2017. “Everest: Towards a Verified, Drop-in Replacement of Https.” In “2nd Summit on Advances in Programming Languages (Snapl).” Special issue, *2nd Summit on Advances in Programming Languages (SNAPL)*,. <https://doi.org/10.4230/LIPIcs.SNAPL.2017.1>.
- Brandt, Michael, and Fritz Henglein. 1998. “Coinductive Axiomatization of Recursive Type Equality and Subtyping.” *Fundamenta Informaticae* 33 (4): 309–38. https://doi.org/10.1007/3-540-62688-3_29.
- Denis, Xavier, Jacques-Henri Jourdan, and Claude Marché. 2022. “Creusot: A Foundry for the Deductive Verification of Rust Programs.” In “Formal Methods and Software Engineering (Icfem).” Special issue, *Formal Methods and Software Engineering (ICFEM)*,. https://doi.org/10.1007/978-3-031-17244-1_6.
- Fu, Peng, and Aaron Stump. 2014. “Self Types for Dependently Typed Lambda Encodings.” In “Rewriting and Typed Lambda Calculi (RTA-Tlca).” Special issue, *Rewriting and Typed Lambda Calculi (RTA-TLCA)*,. <https://homepage.divms.uiowa.edu/~astump/papers/fu-stump-rta-tlca-14.pdf>.
- Ho, Son, and Jonathan Protzenko. 2022. “Aeneas: Rust Verification by Functional Translation.” In “Proceedings of the ACM on Programming Languages (Icfp).” Special issue, *Proceedings of the ACM on Programming Languages (ICFP)* 6. <https://doi.org/10.1145/3547647>.
- Hutchins, DeLesley S. 2010. “Pure Subtype Systems.” In “Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (Popl).” Special issue, *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 287–98. <https://doi.org/10.1145/1706299.1706334>.
- Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, et al. 2009. “seL4: Formal Verification of an OS Kernel.” In “Proceedings of the 22nd ACM Symposium on Operating Systems Principles (Sosp).” Special issue, *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, 207–20. <https://doi.org/10.1145/1629575.1629596>.
- Knop, Dirk. 2026. “As Many Firefox Vulnerabilities Closed in April as in the Previous Two Years.” May. <https://www.heise.de/en/news/As-many-Firefox-vulnerabilities-closed-in-April-as-in-the-previous-two-years-11287175.html>.
- Lattuada, Andrea, Travis Hance, Chancheo Cho, et al. 2023. “Verus: Verifying Rust Programs Using Linear Ghost Types.” In “Proceedings of the ACM on Programming Languages (Oopsla).” Special issue, *Proceedings of the ACM on Programming Languages (OOPSLA)* 7. <https://doi.org/10.1145/3586037>.
- Leroy, Xavier. 2009. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52 (7): 107–15. <https://doi.org/10.1145/1538788.1538814>.
- Lidbury, Charlie. 2024. “Ochre: A Dependently Typed Systems Programming Language.” MEng Individual Project. https://assets.super.so/f890b173-9aa0-4184-8dbd-d0ee94de3ebb/files/00463014-8f18-41dc-81f5-2dccbb91ceae/Ochre_Thesis_Main.pdf.
- Microsoft Security Response Center. 2026. “CVE-2026-31431: COPY_FAIL Vulnerability Enables Linux Root Privilege Escalation.” May. <https://www.microsoft.com/en-us/security/blog/2026/05/01/cve-2026-31431-copy-fail-vulnerability-enables-linux-root-privilege-escalation/>.
- Pasquale, Valentin, and Álvaro García-Pérez. 2026. “Towards the Type Safety of Pure Subtype Systems.” In “34th EACSL Annual Conference on Computer Science Logic (Csl).” Special issue, *34th EACSL Annual Conference on Computer Science Logic (CSL)*, Leibniz International Proceedings in Informatics (LIPIcs), vol. 363 : 37:1–16. <https://doi.org/10.4230/LIPIcs.CSL.2026.37>.
- Protzenko, Jonathan, Jean-Karim Zinzindohoué, Aseem Rastogi, et al. 2017. “Verified Low-Level Programming Embedded in F*.” In “Proceedings of the ACM on Programming Languages (Icfp).” Special issue, *Proceedings of the ACM on Programming Languages (ICFP)* 1. <https://doi.org/10.1145/3110261>.
- Sumner, Jarred. 2026. “Rewrite Bun from Zig to Rust.”. <https://github.com/oven-sh/bun/pull/30412>.

- Taelin, Victor. 2021a. “Beyond Inductive Datatypes: Exploring Self Types.”. <https://github.com/HigherOrderCO/Kind/blob/5a6a5e2f2cff35f77176009dfb4c2f1a1bac2799/blog/1-beyond-inductive-datatypes.md>.
- Taelin, Victor. 2021b. “Kind: A Modern Proof Language.”. <https://github.com/HigherOrderCO-archive/Kind-Legacy/blob/5a6a5e2/base/Fin.kind>.
- Xi, Hongwei. 2017. “Applied Type System: An Approach to Practical Programming with Theorem-Proving.” In “Journal of Functional Programming.” Special issue, *Journal of Functional Programming*,. <https://www.cs.bu.edu/~hwxi/academic/papers/JFPats.pdf>.